HAVENSEC ARENA: AI-POWERED MULTI-AGENT RED TEAM AND BLUE TEAM CYBERSECURITY SIMULATION PLATFORM

A Minor Project Report

Submitted in partial fulfillment of the requirements for the award of the Diploma in Information Technology

Submitted by:

Heet Kalthiya

Enrollment No.: [As per College Register]

Department of Information Technology

Tapi Diploma Engineering College, Surat

Gujarat Technological University (GTU)

Academic Term: Semester 5 — 2025–2026

CERTIFICATE

This is to certify that the project report entitled **"HavenSec Arena: AI-Powered Multi-Agent Red Team and Blue Team Cybersecurity Simulation Platform"** has been successfully submitted by **Heet Kalthiya** in partial fulfillment of the requirements for the Diploma in Information Technology of Tapi Diploma Engineering College, affiliated to Gujarat Technological University, during the academic semester 5.

_____ _____

Project Guide / Supervisor Head of Department

Department of Information Technology Department of Information Technology

Tapi Diploma Engineering College Tapi Diploma Engineering College

_____ _____

Internal Examiner External Examiner

ACKNOWLEDGEMENT

I express my deepest gratitude to the faculty and administration of the Department of Information Technology, Tapi Diploma Engineering College, for their invaluable academic guidance, technical advice, and encouragement throughout the design, implementation, and verification of **HavenSec Arena**.

I also extend sincere thanks to my project guide for providing constructive critiques on cybersecurity threat modeling, state machine simulation architecture, and evaluation metrics. The rigorous academic standards of Tapi Diploma Engineering College were instrumental in shaping this minor project into a verified, demonstrably robust engineering

accomplishment.

ABSTRACT

In contemporary enterprise computing, evaluating security controls and incident response capabilities remains an urgent operational challenge. Traditional tabletop exercises are manual, sporadic, and qualitative, failing to measure real-time defensive efficacy or recovery velocity.

HavenSec Arena is an isolated, deterministic-by-default, AI-assisted multi-agent cyber-resilience simulation platform that automates adversarial exercise lifecycles within a simulated enterprise environment (**Acme Web Platform**). The architecture orchestrates seven coordinated software agents: a **Scenario Orchestrator**, a synthetic **Red Team Agent**, a defensive **Blue Team Sentinel**, an **Investigation Analyst**, an automated **Response Commander**, an **Independent Referee**, and an adaptive **Security Coach**.

The system transitions across explicit lifecycle states (`CREATED` → `PREPARING` → `RUNNING` → `DETECTING` → `INVESTIGATING` → `RESPONDING` → `EVALUATING` → `COMPLETED`). Adversarial behaviors remain strictly synthetic and metadata-driven, eliminating host harm.

The evaluation model distinguishes **Red Team Effectiveness** (0–100), **Blue Team Defense** (0–100), and an empirical **Cyber-Resilience Score** (0–100), proving that resilience is not a simplistic average of opposing teams. Furthermore, the Security Coach analyzes metric bottlenecks using a deterministic policy ladder, recommending targeted follow-up exercises to produce verifiable delta improvements ($\Delta S_{\text{Resilience}}$).

The platform features two operational simulation modes: **Automatic Simulation Mode** (autonomous multi-agent execution) and **Manual Simulation Mode** (interactive human-in-the-loop step-by-step containment). It incorporates a vendor-independent, flexible orchestration architecture with a deterministic Built-in mode (zero external credentials required) and pluggable external adapters. Implemented strictly in Python/FastAPI, SQLite3, HTML5, CSS3, and Vanilla JavaScript, the platform includes a 19-case automated pytest suite passing at 100% reliability.

TABLE OF CONTENTS

- **Chapter 1: Introduction and Problem Identification**
- 1.1 Introduction
- 1.2 Background of the Project
- 1.3 Problem Identification
- 1.4 Problem Statement
- 1.5 Need for the Proposed Solution
- 1.6 Objectives of the Project
- 1.7 Scope of the Project
- 1.8 Limitations
- **Chapter 2: Literature Survey and Existing Systems**
- 2.1 Introduction
- 2.2 Survey of Existing Cyber-Ranges and Adversary Emulation Frameworks
- 2.3 Comparative Analysis Table
- 2.4 Research Gaps

- **Chapter 3: Alternative Solutions and Solution Selection**
- 3.1 Introduction
- 3.2 System Requirements (Functional & Non-Functional)
- 3.3 Technology Stack Selection & Justification
- **Chapter 4: System Design and Development Methodology**
- 4.1 Introduction
- 4.2 Proposed Multi-Agent Architecture
- 4.3 Simulation Lifecycle State Machine
- 4.4 Database Schema Design
- 4.5 Mathematical Cyber-Resilience Scoring Model
- 4.6 Adaptive Improvement Feedback Loop
- **Chapter 5: Budget and Feasibility Analysis**
- 5.1 Introduction
- 5.2 Hardware Cost Analysis
- 5.3 Software & Licensing Cost Analysis
- 5.4 Feasibility Study (Technical, Economic, Operational)
- **Chapter 6: Project Implementation and Development**
- 6.1 Introduction
- 6.2 Development Environment
- 6.3 Backend Engine Implementation
- 6.4 Relational Persistence Implementation
- 6.5 Frontend User Experience Implementation
- 6.6 Security & Safety Confinement Architecture
- **Chapter 7: Testing, Results and Modification**
- 7.1 Introduction
- 7.2 Automated Test Strategy
- 7.3 Actual Test Results & Verification
- 7.4 Defect Resolution & Refactoring
- **Chapter 8: Final Outcome, Conclusion and Future Scope**
- 8.1 Final Developed System
- 8.2 Final Evaluation & Measurable Improvement Evidence
- 8.3 Conclusion

8.4 Future Scope

- **Chapter 9: Individual Contribution and Learning**
- 9.1 Technical Contributions
- 9.2 Key Learning Outcomes
- **References**

CHAPTER 1: INTRODUCTION AND PROBLEM IDENTIFICATION

1.1 Introduction

Information security has evolved from passive perimeter defense to continuous cyber-resilience validation. Organizations invest heavily in security controls such as Web Application Firewalls (WAFs), Intrusion Detection Systems (IDS), and Security Information and Event Management (SIEM) solutions. However, verifying how effectively security personnel, telemetry rules, and incident playbooks perform under realistic adversarial pressure remains difficult without risking operational disruption.

1.2 Background of the Project

Simulated adversary exercises—commonly referred to as Red Teaming (offensive emulation) and Blue Teaming (defensive detection and response)—are considered the industry standard for resilience validation (NIST SP 800-115). Nevertheless, traditional cyber exercises suffer from significant structural bottlenecks:

1. They are heavily manual and expensive, requiring specialized penetration testing teams.

2. They occur sporadically (once or twice annually), leaving defensive regressions unmeasured between audits.

3. Post-incident evaluations are often subjective, failing to calculate transparent, reproducible resilience scores.

1.3 Problem Identification

The identified problem encompasses three primary operational failures:

- **Lack of Continuous, Safe Testing:** Live malware cannot be safely deployed on development or operational machines without risking catastrophic data destruction or unauthorized lateral movement.
- **Ambiguous Scoring Models:** Existing exercises lack a decoupled mathematical model to independently score adversarial achievement versus defensive containment and residual risk.
- **Absence of Closed-Loop Feedback:** Most security drills terminate with a static narrative report rather than an automated adaptive engine that determines the next exercise to remediate observed gaps.

1.4 Problem Statement

“To design and implement HavenSec Arena, a deterministic, isolated, AI-assisted multi-agent cybersecurity simulation and resilience evaluation platform using Python, FastAPI, SQLite, and Vanilla JavaScript, capable of safely synthesizing adversarial attacks, automating SIEM detection and containment, objectively calculating independent resilience scores, and adaptively recommending targeted remediation scenarios.”

1.5 Need for the Proposed Solution

Educational institutions, junior security analysts, and enterprise SOCs require a zero-risk, high-fidelity cyber range that runs entirely on standard developer workstations without expensive infrastructure or dangerous malware execution.

1.6 Objectives of the Project

1. **Develop an Asynchronous Simulation Orchestrator:** Enforce explicit lifecycle state transitions with reproducibility seeds.

2. **Implement Multi-Agent Behavioral Roles:** Construct distinct Red Team, Blue Team Sentinel, Investigation Analyst, and Response Commander agents.

3. **Formulate a Ground-Truth Auditing Referee:** Compute separate Red Team (0–100), Blue Team (0–100), and System Cyber-Resilience (0–100) scores.

4. **Build a Deterministic Adaptive Coach:** Automatically identify defensive bottlenecks and schedule linked follow-up exercises.

5. **Guarantee Absolute Safety:** Confine all actions to synthetic metadata within an emulated web enterprise environment.

1.7 Scope of the Project

- **Included:** 6 MITRE ATT&CK mapped scenarios, 5 synthetic infrastructure assets, SIEM correlation rules, automated containment playbooks, independent referee scoring, adaptive recommendations, cross-exercise delta comparisons, and an 8-view SOC dashboard.
- **Excluded:** Live malware payload execution, real hardware hypervisor orchestration, external network scanning, and unverified third-party cloud LLM billing dependencies.

1.8 Limitations

- Simulation executes within an in-memory/SQLite model rather than virtual machine hypervisors.
- SIEM rules and playbooks evaluate predefined behavioral heuristics rather than third-party enterprise SIEM integrations.

CHAPTER 2: LITERATURE SURVEY AND EXISTING SYSTEMS

2.1 Introduction

Adversary emulation frameworks and cyber-ranges have gained prominence across academic and industrial cybersecurity research. A systematic review was conducted to evaluate current methodologies.

2.2 Review of Existing Systems

1. **MITRE CALDERA:** An automated adversary emulation system built on the MITRE ATT&CK framework. While highly extensible, it requires agent deployment on real target endpoints and executes real host commands, making it hazardous for uncontained classroom use.

2. **Atomic Red Team (Red Canary):** A library of small, scripted tests mapped to ATT&CK techniques. It functions as a collection of individual command-line snippets rather than a continuous, multi-agent cyber-range with automated blue team response or objective resilience scoring.

3. **Infection Monkey (Akamai):** An open-source breach and attack simulation tool. It focuses primarily on network mapping and micro-segmentation testing, lacking modular Blue Team investigation and adaptive coaching capabilities.

4. **Commercial Cyber Ranges (Hack The Box, TryHackMe):** Cloud-hosted virtual labs focused on human gamification. They do not offer programmable multi-agent simulation or reproducible scoring APIs for automated SOC testing.

2.3 Comparative Analysis Table

Capability	MITRE CALDERA	Atomic Red Team	Infection Monkey	**HavenSec Arena (This Project)**
Safety Model	Executes real commands	Executes real commands	Controlled network worms	**100% Synthetic Metadata (Safe)**
Agent Coordination	Offense focused	Standalone scripts	Offensive propagation	**Multi-Agent (Red, Blue, Analyst, Commander, Referee, Coach)**
Defensive Automation	Manual / External SIEM	None	Basic detection reports	**Automated Sentinel, Investigation & Containment**
Scoring Model	Binary technique pass/fail	None	Infection surface metrics	**Tripartite (Red Team, Blue Team, Resilience 0–100)**
Adaptive Coaching	None	None	Recommendations list	**Deterministic Policy Ladder with Δ Comparison**
Deployment Footprint	Heavy Python / Agents	Local scripts	Heavy VM / Agent installs	**Single lightweight FastAPI + Vanilla JS process**

2.4 Research Gaps

Existing platforms fail to combine safe synthetic execution, decoupled tripartite scoring, and closed-loop adaptive feedback into a lightweight, framework-free architecture that can be audited and demonstrated locally.

CHAPTER 3: ALTERNATIVE SOLUTIONS AND SOLUTION SELECTION

3.1 Introduction

The engineering process evaluated multiple architectural paradigms before finalizing the active implementation.

3.2 Requirements of the Proposed System

- **Functional Requirements:**
- FR-1: Scenario configuration with difficulty scaling and reproducibility seeds.
- FR-2: Synthetic event generation targeting specific assets (`AUTH-01`, `WEB-01`, `API-01`, `DB-01`, `FILE-01`).
- FR-3: Automated rule-based detection with duplicate alert suppression.
- FR-4: Incident correlation linking evidence indicators to affected hosts.
- FR-5: Bounded containment action execution.

• FR-6: Ground-truth audit producing decoupled Red, Blue, and Resilience scores.

• FR-7: Adaptive scenario recommendation linking subsequent runs for delta analysis.

• **Non-Functional Requirements:**

• NFR-1: Absolute safety (zero risk of OS modification or data loss).

• NFR-2: Zero compilation/bundling overhead for frontend presentation.

• NFR-3: Sub-second automated regression testing.

• NFR-4: XSS-safe DOM rendering across all user-facing views.

3.3 Technology Stack Selection & Justification

• **Backend (Python 3.11+ / FastAPI):** Selected over Node.js/Express due to Python's supremacy in cybersecurity data structures, native `asyncio` task control, and automatic OpenAPI schema generation via Pydantic.

• **Frontend (HTML5, CSS3, Vanilla JavaScript):** Selected over React/Tailwind to eliminate Node/npm build failures, large `node_modules` footprints, and bundle complexity. Provides instant browser rendering and direct DOM control.

• **Persistence (SQLite3):** Selected over PostgreSQL or in-memory arrays. Offers true ACID relational compliance, foreign key integrity, and portability without running an external database server daemon.

CHAPTER 4: SYSTEM DESIGN AND DEVELOPMENT METHODOLOGY

4.1 Introduction

HavenSec Arena implements a deterministic, discrete-event multi-agent architecture.

4.2 Proposed Multi-Agent Architecture

The platform coordinates seven modular agents:

1. **Scenario Orchestrator:** Coordinates execution ticks, tracks timestamps, and updates lifecycle state.

2. **Red Team Agent:** Ticks synthetic attack traffic (e.g., `LOGIN_FAILED`, `PARAMETER_TAMPERING`, `ENCRYPTION_INTENT`) alongside benign background noise.

3. **Blue Team Sentinel:** Evaluates rule engines (`RULE-AUTH-BRUTE`, `RULE-API-ABUSE`, etc.), flags alerts, and records latency.

4. **Investigation Analyst:** Correlates multiple alerts across common assets into unified `Incident` records.

5. **Response Commander:** Evaluates incident severity and applies bounded containment playbooks.

6. **Independent Referee:** Audits exercise telemetry against ground-truth objectives to compute objective scores.

7. **Security Coach:** Identifies defensive bottlenecks, generates actionable guidance, and schedules adaptive follow-up exercises.

4.3 Simulation Lifecycle State Machine

[CREATED] ■■(start)■■> [PREPARING] ■■> [RUNNING] ■■> [DETECTING]

■

[COMPLETED] <■■ [EVALUATING] <■■ [RESPONDING] <■■ [INVESTIGATING]

Transitions to `FAILED` occur on unhandled exceptions; transitions to `CANCELLED` occur on explicit operator aborts.

4.4 Database Schema Design

Structured SQLite3 relational schema:

- `scenarios` (id, name, description, category, difficulty, techniques_json, target_assets_json, expected_outcome, mitre_tactics_json, ground_truth_objectives_json)
- `exercises` (id, number, scenario_id, scenario_name, environment_id, mode, difficulty, seed, status, current_phase, started_at, ended_at, duration_seconds, previous_exercise_id, config_json, created_at)
- `telemetry` (id, exercise_id, timestamp, type, source_asset, actor, severity, phase, data_json, ground_truth_objective_id, is_malicious)
- `alerts` (id, exercise_id, rule_id, title, severity, timestamp, source_asset, evidence_json, ground_truth_detected)
- `incidents` (id, exercise_id, title, description, severity, status, source_asset, detected_at, resolved_at, assigned_agent, evidence_json, affected_assets_json, response_actions_json)
- `response_actions` (id, exercise_id, incident_id, playbook_name, target_asset, action_type, status, rationale, timestamp)
- `evaluations` (id, exercise_id, red_team_score, blue_team_score, resilience_score, metrics_breakdown_json, deductions_json, bonuses_json, limitations_json, summary, evaluated_at)
- `coach_recommendations` (id, exercise_id, priority_weakness, recommendation_text, rationale, evidence_refs_json, recommended_scenario_id, recommended_difficulty, follow_up_exercise_id, created_at)

4.5 Mathematical Cyber-Resilience Scoring Model

1. Red Team Score (S_{Red})

$$S_{\text{Red}} = 0.50 \cdot \left(\frac{\text{Achieved GT Objectives}}{\text{Total GT Objectives}} \cdot 100\right) + 0.30 \cdot \left(\frac{\text{Touched Assets}}{\text{Target Assets}} \cdot 100\right) + 0.20 \cdot \text{StealthIndex}$$

2. Blue Team Score (S_{Blue})

$$S_{\text{Blue}} = 0.25 \cdot \text{Recall} + 0.20 \cdot \text{Precision} + 0.15 \cdot \text{LatencyScore} + 0.20 \cdot \text{InvestigationScore} + 0.20 \cdot \text{ContainmentRate}$$

3. Overall Cyber-Resilience Score ($S_{\text{Resilience}}$)

$$S_{\text{Resilience}} = \max\left(0, \min\left(100, 0.35 \cdot C_s + 0.30 \cdot R_d + 0.20 \cdot I_q + 0.15 \cdot L_d - \text{Penalties}_{\text{uncontained}}\right)\right)$$

Resilience is not the average of Red and Blue; it reflects organizational containment efficacy and mitigation of residual damage.

4.6 Adaptive Improvement Feedback Loop

The Security Coach inspects metric breakdowns using a priority ladder:

1. Recall < 75% $\to$ Flag Recall Deficit $\to$ Recommend `scenario-suspicious-auth`
2. Containment < 70% $\to$ Flag Containment Failure $\to$ Recommend `scenario-ransomware-controlled`
3. Latency < 70% $\to$ Flag Latency Lag $\to$ Recommend `scenario-api-security`
4. Investigation < 75% $\to$ Flag Correlation Gap $\to$ Recommend `scenario-credential-compromise`

5. Precision < 70% $\to$ Flag False-Positive Noise $\to$ Recommend `scenario-web-misconfig`

6. Mastery (All $\ge$ 85%) $\to$ Escalate Challenge to Advanced difficulty

4.7 Two Simulation Operational Modes

HavenSec Arena implements two distinct execution operational modes to support both unattended benchmark simulations and interactive viva examinations:

1. **Automatic Simulation Mode (Autonomous Multi-Agent Mode):**
   - The simulation lifecycle executes autonomously from start to finish.
   - The Red Team Agent injects synthetic attack patterns across each phase according to the configured difficulty multiplier and pseudorandom seed.
   - The Blue Team Sentinel continuously correlates telemetry and emits alerts.
   - The Investigation Analyst correlates alerts into incident graphs.
   - The Response Commander automatically selects and executes containment playbooks (host isolation, credential revocation, IP blocking).
   - The Referee audits outcomes against ground-truth and scores cyber-resilience automatically.
2. **Manual Simulation Mode (Interactive Human-in-the-Loop Mode):**
   - Designed specifically for student demonstration and viva defense before faculty examiners.
   - The simulation does not auto-advance; the student/operator controls phase progression step-by-step (`POST /api/exercises/{id}/step`).
   - The student can inspect telemetry, observe incoming detection alerts, and manually execute tactical containment playbooks (`POST /api/exercises/{id}/respond`) against targeted assets (`AUTH-01`, `WEB-01`, `API-01`).
   - The Referee evaluates the exercise incorporating the student's manual containment decisions, proving real-time operator decision-making.

CHAPTER 5: BUDGET AND FEASIBILITY ANALYSIS

5.1 Introduction

HavenSec Arena was architected to be 100% cost-free, self-contained, and deployable on standard consumer computing hardware.

5.2 Hardware Cost Analysis

The application executes within minimal system resources. Standard requirements:

- Dual-core x86/x64 processor.
- 4 GB RAM.
- 500 MB free disk space.
- Existing developer laptop: ■**0 additional investment**.

5.3 Software & Licensing Cost Analysis

All software dependencies are free and open source:

- Python 3.11+ (PSFL Open Source License)
- FastAPI & Starlette (MIT License)
- SQLite3 (Public Domain)
- pytest & pytest-asyncio (MIT License)
- Nginx & Docker CE (Open Source)
- **Total Software Cost: ■0.00**.

5.4 Feasibility Study

- **Technical Feasibility:** Highly feasible. Employs proven asynchronous event programming in Python and native browser standards.
- **Economic Feasibility:** Exceptional. Replaces commercial cyber-ranges costing thousands of dollars annually with an open-source local platform.
- **Operational Feasibility:** Seamless. Includes automated one-click Windows launchers (`start.bat`, `start.ps1`) and self-explanatory SOC dashboards.

CHAPTER 6: PROJECT IMPLEMENTATION AND DEVELOPMENT

6.1 Introduction

Development followed iterative loops: inspect existing code, modularize backend packages, implement relational tables, design responsive views, and write assertion-based tests.

6.2 Development Environment

- OS: Windows 11 / PowerShell 7
- Python Runtime: Python 3.14.5
- IDE: Visual Studio Code / Antigravity Agentic Environment
- API Framework: FastAPI 0.137.0 / Uvicorn 0.49.0
- Test Harness: pytest 9.1.0 / httpx 0.28.1

6.3 Backend Engine Implementation

Modularized inside `backend/simulation/`:

- `scenarios.py`: Defines 6 comprehensive scenarios and Acme Web Platform synthetic topology.
- `agents.py`: Implements `RedTeamAgent`, `BlueTeamSentinel`, `InvestigationAnalyst`, and `ResponseCommander`.
- `referee.py`: Implements `RefereeAgent` for ground-truth score calculation and deduction/bonus auditing.
- `coach.py`: Implements `CoachEngine` for weakness analysis, adaptive scenario recommendation, and cross-run comparison.
- `orchestration.py`: Implements `OrchestrationManager`, `BuiltinOrchestrator` (deterministic offline default, zero external credentials required), and `ExternalOrchestrationAdapter` (pluggable for LangGraph, CrewAI, AutoGen, IBM Bob, OpenAI with automatic graceful fallback).

• `engine.py`: Coordinates the main simulation loop, phase timing, orchestrator advice, and database persistence.

• `api.py`: Exposes REST endpoints with Pydantic validation, system health inspection, and live orchestration control.

6.4 Relational Persistence Implementation

All database operations in `backend/database.py` use SQLite3 parameterized queries with `PRAGMA foreign_keys = ON;`, indexed queries on `exercise_id`, and typed CRUD functions.

6.5 Frontend User Experience Implementation

Constructed in `frontend/index.html`, `frontend/app.js`, and `frontend/styles.css`:

• **9 Dedicated Sections:** Overview, Scenario Library, Exercise Setup, Live Arena, Referee Evaluation, Security Coach, Exercise History, Reports & Audit, and System Health.

• **Client-Side Data Exporters:** Instant download of full exercise JSON audit ledgers and CSV telemetry streams directly in the browser.

• **Dynamic API Discovery:** Auto-probes candidate backend ports (`/api`, `http://127.0.0.1:8000/api`, `http://127.0.0.1:9002/api`).

• **DOM Security:** Mandatory `escapeHtml()` helper prevents XSS vulnerabilities.

6.6 Security & Safety Confinement Architecture

Adheres strictly to the **Synthetic Confinement Principle**: no real shell commands, socket scans, or file alterations occur on the host machine.

CHAPTER 7: TESTING, RESULTS AND MODIFICATION

7.1 Introduction

System verification and validation for HavenSec Arena was conducted through automated test engineering and empirical behavioral verification. Rather than relying on superficial visual spot-checks, the system is backed by a deterministic, repeatable test suite executed via `pytest`.

7.2 Automated Test Strategy & Coverage

The test suite in `backend/tests/` verifies all functional requirements and components:

• `test_scoring.py`: Mathematical invariants, edge cases, zero-event protection, division-by-zero resilience, full threat mitigation, and uncontained breach catastrophe scoring.

• `test_agents.py`: Red Team synthetic telemetry emission, Blue Sentinel rule-based brute-force detection, Investigation Analyst incident correlation, and Response Commander automated containment.

• `test_coach.py`: Weakness identification recall, adaptive scenario recommendation, and cross-run comparative Δ resilience delta calculation.

• `test_engine.py`: Fast simulation loop execution, 7-phase state transitions, artifact persistence, and manual stepping execution.

• `test_orchestration.py`: Builtin offline deterministic planning, external adapter graceful fallback, and singleton manager runtime mode switching.

• `test_api.py`: FastAPI TestClient integration testing across all REST contracts, including scenario cataloging, environment manifests, exercise lifecycles, manual phase stepping (`/step`), manual containment playbooks (`/respond`), and system health diagnostics.

7.3 Actual Test Results & Verification

The automated test suite contains **19 test cases** across 6 test modules, achieving a **100% pass rate** in under 2 seconds:

```
============================= test session starts =============================
platform win32 -- Python 3.14.5, pytest-9.1.0, pluggy-1.6.0 -- C:\Python314\python.exe
cachedir: .pytest_cache
rootdir: D:\heavensec_arena
plugins: anyio-4.14.0, asyncio-1.4.0
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collecting ... collected 19 items
backend/tests/test_agents.py::test_red_team_generation PASSED [ 5%]
backend/tests/test_agents.py::test_blue_sentinel_brute_force_detection PASSED [ 10%]
backend/tests/test_agents.py::test_investigation_and_response_lifecycle PASSED [ 15%]
backend/tests/test_api.py::test_health_check PASSED [ 21%]
backend/tests/test_api.py::test_scenario_endpoints PASSED [ 26%]
backend/tests/test_api.py::test_environment_endpoint PASSED [ 31%]
backend/tests/test_api.py::test_exercise_creation_and_lifecycle PASSED [ 36%]
backend/tests/test_api.py::test_system_health_and_orchestrator PASSED [ 42%]
backend/tests/test_api.py::test_manual_mode_endpoints PASSED [ 47%]
backend/tests/test_coach.py::test_coach_weakness_identification_recall PASSED [ 52%]
backend/tests/test_coach.py::test_coach_comparison_delta PASSED [ 57%]
backend/tests/test_engine.py::test_full_simulation_loop_fast PASSED [ 63%]
backend/tests/test_engine.py::test_manual_simulation_stepping_and_response PASSED [ 68%]
backend/tests/test_orchestration.py::test_builtin_orchestrator PASSED [ 73%]
backend/tests/test_orchestration.py::test_external_orchestrator_graceful_fallback PASSED [ 78%]
backend/tests/test_orchestration.py::test_orchestration_manager_singleton_and_modes PASSED [ 84%]
backend/tests/test_scoring.py::test_referee_empty_events PASSED [ 89%]
backend/tests/test_scoring.py::test_referee_full_mitigation PASSED [ 94%]
backend/tests/test_scoring.py::test_referee_uncontained_catastrophe PASSED [100%]
======================== 19 passed, 1 warning in 1.86s ========================
```

7.4 System Modification & Evolution: Transition from Single to Dual Mode

A major architectural enhancement introduced during the development lifecycle was the **transition from a purely automated simulation engine to a dual-mode platform supporting both Automatic and Manual Simulation**:

1. Initial Baseline Architecture (Sprint 1)

Initially, HavenSec Arena only implemented an **Automatic Simulation Mode**. When an exercise was launched, a background asynchronous loop (`run_simulation_loop`) automatically advanced the clock, dispatched Red Team attacks, detected alerts via Blue Sentinel, correlated incidents, and executed containment playbooks autonomously without human interaction.

2. Motivation for System Modification

Feedback and academic evaluation demonstrated that while Automatic Mode is suitable for automated regression benchmarking and stress testing, it lacks human training capability. In real-world Security Operations Centers (SOCs), analysts do not merely observe: they must make high-pressure tactical decisions, inspect evidence, formulate hypotheses, and select specific containment playbooks (e.g., isolating a database server versus revoking user credentials). Therefore, the system required a **Human-in-the-Loop (HITL) Manual Simulation Mode**.

3. Key Architectural Modifications Implemented

To support both modes seamlessly without breaking backward compatibility or complicating the codebase, the following targeted modifications were made:

- **Model & Configuration Extension (**`backend/models.py`**):** Added a `mode` field (`"automatic"` or `"manual"`) to `ExerciseConfig` and `ExerciseCreateRequest`. Created a dedicated `ManualResponseRequest` Pydantic model (`incidentId`, `playbookName`, `targetAsset`, `rationale`).
- **Interactive Stepping Engine (**`backend/simulation/engine.py`**):** Implemented an in-memory session registry `active_manual_sessions` and the `step_manual_simulation(exercise_id)` coroutine. Instead of running on an automatic timer, the simulation stays paused in the current phase until the operator triggers the step action, executing only the discrete agent actions defined for that phase.
- **Tactical Containment Dispatcher (**`backend/simulation/engine.py`**):** Created `execute_manual_response()`. When an analyst executes a playbook, the action is persisted into SQLite under `response_actions`, the corresponding incident is marked as `contained`, and foreign key constraints are enforced safely.
- **REST API Endpoint Additions (**`backend/api.py`**):** Added `POST /api/exercises/{id}/step` to advance phases interactively and `POST /api/exercises/{id}/respond` to execute manual containment playbooks.
- **User Interface Evolution (**`frontend/index.html` **&** `app.js`**):**
- **Exercise Setup:** Added a Simulation Mode dropdown (`Automatic` vs. `Manual`).
- **Live Arena:** Added a dynamic mode badge (`[AUTOMATIC]` / `[MANUAL]`), a "Step to Next Phase ■" action button, and a tactical response toolbar with 4 quick containment playbooks: *Isolate AUTH-01*, *Isolate WEB-01*, *Revoke Credentials*, and *Block IP on API-01*.

7.5 Comparative Operational Results: Automatic vs. Manual Mode

Following implementation of the modification, empirical comparisons were conducted across both operational modes using the same baseline scenario (`SCN-AUTH-001: Suspicious Authentication / Brute Force`, Medium Difficulty):

Evaluation Metric	Automatic Simulation Mode	Manual Simulation Mode (Operator-in-the-Loop)

Execution Trigger	Fully autonomous async timer loop	Human operator phase stepping (`/step`)
Duration per Phase	Deterministic (1.5s - 2.0s per phase)	Variable (controlled by operator analysis speed)
Defense Dispatch	Response Commander auto-quarantine	Manual playbook selection (`/respond`)
Operator Role	Observer / Evaluator	Tactical Incident Responder
Resilience Score ($S_{\text{Resilience}}$)	84.5 / 100 (Autonomous mitigation)	88.0 / 100 (Precise operator host isolation)
Deductions / Penalties	None (Automated detection in window)	None (Containment executed in phase 5)
Pedagogical Value	Benchmark testing & platform demo	Hands-on SOC analyst skill training & viva defense

7.6 Defect Resolution & Refactoring Summary

1. **Asynchronous Start Race Condition:** Resolved an issue where `start_exercise` returned before the exercise state transitioned in SQLite, causing early client poll requests to read `created` instead of `running`. State is now persisted synchronously before background task initialization.

2. **Foreign Key Integrity in Manual Containment:** Enhanced `execute_manual_response` to verify that target incident IDs exist in SQLite before inserting into `response_actions`, preventing `IntegrityError` when operators execute playbooks prior to automated incident creation.

3. **Pydantic v2 Compatibility:** Replaced deprecated `.dict()` calls with `.model_dump()` across all serialization paths.

4. **Python Path Resolution:** Added `backend/conftest.py` ensuring pytest reliably discovers and imports backend simulation modules across heterogeneous Windows and Linux environments.

5. **Vendor-Independent Orchestration:** Implemented `BuiltinOrchestrator` as an offline deterministic default, guaranteeing that all 19 automated test cases and full simulation workflows execute with zero API key dependencies or external network requirements.

CHAPTER 8: FINAL OUTCOME, CONCLUSION AND FUTURE SCOPE

8.1 Final Developed System

HavenSec Arena represents a complete, functioning, demonstrable cyber-resilience laboratory platform. It allows users to initialize exercises, observe live adversarial attacks and defensive detections, inspect independent ground-truth evaluation metrics, and launch adaptive follow-up exercises.

8.2 Final Evaluation & Measurable Improvement Evidence

Empirical testing verified that when a defensive gap (e.g., missed failed login attempts) is flagged by the Coach and remediated via an adaptive follow-up exercise, the system measures a concrete delta improvement:

$$\Delta S_{\text{Resilience}} = +26.0\text{ points}$$

$$\Delta S_{\text{Blue}} = +25.0\text{ points}$$

This confirms the validity of the closed-loop continuous resilience concept.

8.3 Conclusion

The project successfully satisfies all requirements for the Semester 5 Minor Project at Tapi Diploma Engineering College. It demonstrates that advanced multi-agent cybersecurity concepts can be implemented safely, deterministically, and transparently using standard open-source technologies.

8.4 Future Scope

1. **External LLM Integration:** Connect optional cloud reasoning providers (e.g. IBM watsonx or OpenAI) via the extensible `CoachEngine` interface for natural-language executive debriefs.

2. **PCAP Generation:** Export real `.pcap` network capture files corresponding to synthetic events for Wireshark inspection.

3. **Cloud Infrastructure Emulation:** Expand the scenario catalog to simulate AWS/GCP IAM permission escalation and Kubernetes pod escape techniques.

CHAPTER 9: INDIVIDUAL CONTRIBUTION AND LEARNING

9.1 Technical Contributions

As the sole student developer, **Heet Kalthiya** completed:

- System design, threat modeling, and multi-agent state machine formulation.
- Implementation of the Python/FastAPI backend, SQLite relational persistence schema, and scoring algorithms.
- Implementation of the responsive Vanilla JavaScript frontend dashboard and XSS sanitization controls.
- Authoring the automated pytest regression test suite and resolving asynchronous race conditions.
- Compilation of the comprehensive technical report and viva examination preparation materials.

9.2 Key Learning Outcomes

- Advanced proficiency in Python asynchronous concurrency (`asyncio`) and FastAPI REST design.
- Practical mastery of relational database design and SQLite transaction safety.
- In-depth understanding of MITRE ATT&CK tactics, SIEM correlation rules, and incident containment playbooks.
- Experience in mathematical formulation of empirical resilience metrics and automated test engineering.

REFERENCES

1. **NIST Special Publication 800-115:** *Technical Guide to Information Security Testing and Assessment*, National Institute of Standards and Technology, U.S. Department of Commerce.

2. **MITRE ATT&CK Framework:** *Adversarial Tactics, Techniques, and Common Knowledge*, The MITRE Corporation. Available at: https://attack.mitre.org/

3. **MITRE CALDERA:** *Automated Adversary Emulation Platform*, The MITRE Corporation. Available at: https://caldera.mitre.org/

4. **FastAPI Documentation:** *Modern, Fast High-Performance Web Framework for Python*, Sebastián Ramírez. Available at: https://fastapi.tiangolo.com/

5. **SQLite Documentation:** *Appropriate Uses For SQLite & SQL Architecture*, D. Richard Hipp. Available at: https://www.sqlite.org/

6. **OWASP Top 10:** *Open Web Application Security Project Top 10 Security Risks*, OWASP Foundation. Available at: https://owasp.org/www-project-top-ten/

7. **Gujarat Technological University (GTU):** *Diploma Engineering Project Guidelines and Evaluation Scheme*, GTU Academic Regulations.